

Tabuleiro de Xadrez Eletrônico com Integração a *Chess Engine*

Relatório da Semana 3

Grupo W

PCS3732 — Laboratório de Processadores

Agosto de 2026

Resumo

A terceira semana do projeto foi dedicada à camada de baixo nível e à interface com o usuário. O processo em C foi reescrito de forma modular e ganhou uma segunda camada de entrada — um teclado matricial 4×4 em que os lances são digitados —, intercambiável com a matriz de *reed switches* e falando exatamente o mesmo protocolo IPC com a aplicação Python. Esse é o **plano B** do projeto: a montagem do tabuleiro instrumentado avançou, mas não deve ficar pronta no prazo, e com o teclado o sistema inteiro (Stockfish, Lichess, GUI, roque, capturas) continua jogável. Em paralelo, as escolhas que antes só existiam na linha de comando passaram a ter interface gráfica, e a suíte automatizada ganhou 30 verificações novas que exercitam o binário em C de ponta a ponta.

Conteúdo

1	Desenvolvimento de Software	2
1.1	Reestruturação do processo em C	2
1.2	Plano B: camada de teclado matricial	4
1.3	Extensão do protocolo IPC: linhas @entry	6
1.4	Interface de configuração	6
1.5	Ações durante a partida	7
2	Desenvolvimento de Hardware	7
3	Evidências de Testes Experimentais	8
3.1	Suíte automatizada	8
3.2	Verificações da camada de teclado	8
3.3	O que essas evidências cobrem	10
3.4	Rastreabilidade requisito × teste	10
4	Documentação e Diagramas	10

5 Estruturação do Relatório Final	12
6 Entrega e Próximos Passos	12

1 Desenvolvimento de Software

1.1 Reestruturação do processo em C

Na semana 2 o código de baixo nível era uma adaptação direta da prova de conceito feita para Arduino: um único arquivo que inicializava a `wiringPi`, varria a matriz e imprimia as mudanças. Nesta semana ele foi reescrito como um programa modular, com responsabilidades separadas em arquivos próprios e uma interface de linha de comando (Tabela 1).

Tabela 1: Módulos do processo em C (`src/`).

Arquivo	Responsabilidade
<code>main.c</code>	CLI: escolhe a camada de entrada e liga-a ao canal IPC
<code>reed_layer.c</code>	Camada 1: varredura da matriz 8×8 e <i>debouncing</i>
<code>keypad_layer.c</code>	Camada 2: teclado matricial 4×4, lances digitados
<code>board.c</code>	Casas do tabuleiro e espelho de ocupação
<code>ipc.c</code>	Serialização dos eventos (<i>stdout</i> ou <i>Named Pipe</i>)
<code>lcd.c</code>	Display 16×2 por I ² C (opcional)
<code>gpio.c</code>	<i>Wrapper</i> da <code>wiringPi</code> (compila também sem ela)
<code>runstate.c</code>	Encerramento limpo em SIGINT/SIGTERM

A decisão de projeto que organiza tudo isso é a de **camadas de entrada intercambiáveis**: as duas camadas emitem exatamente os mesmos eventos, então a aplicação Python não sabe — nem precisa saber — qual delas está do outro lado. Trocar de uma para a outra é trocar uma opção de linha de comando (`--input reed` ou `--input keypad`), sem tocar em nada da lógica de jogo.

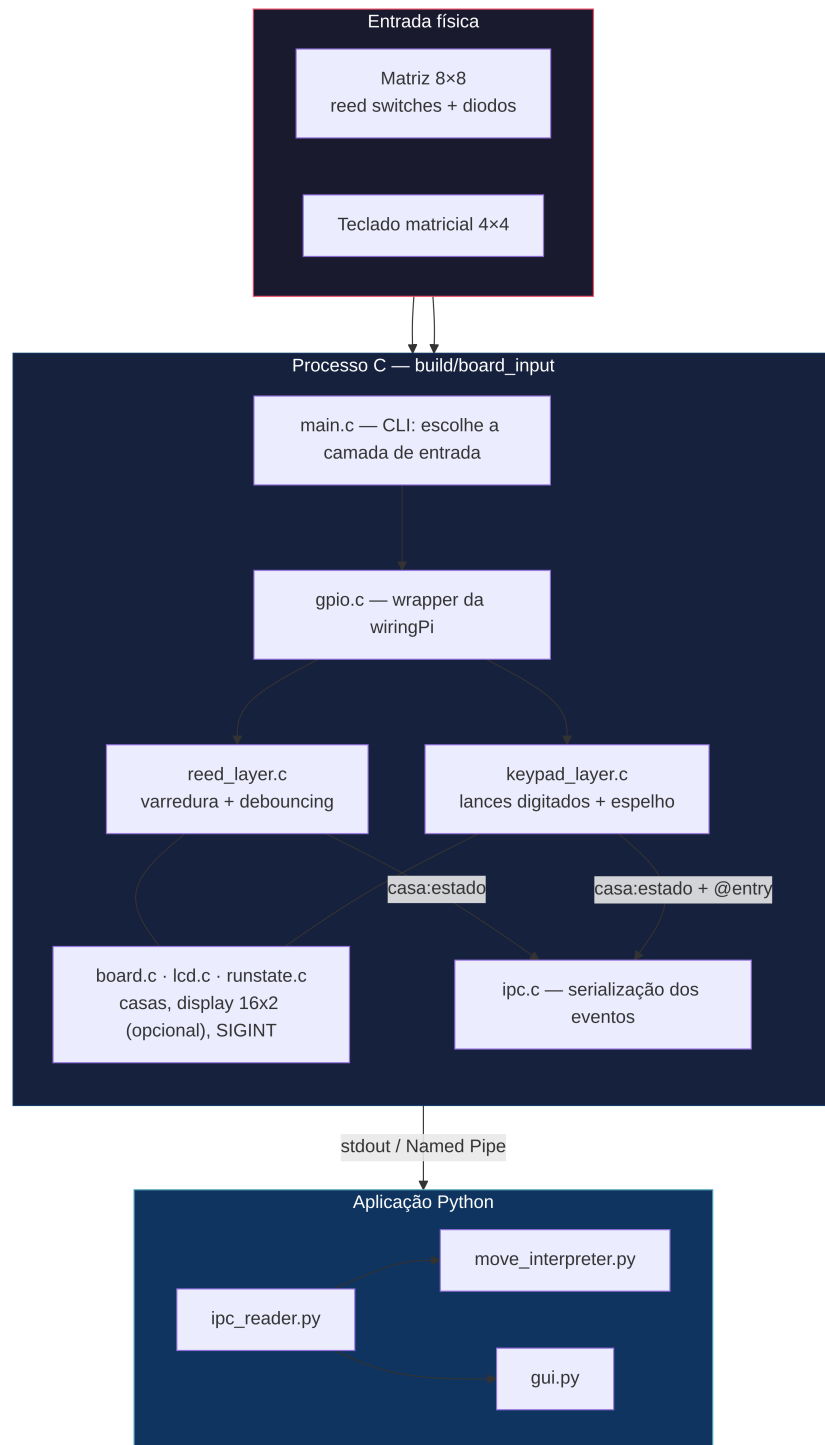


Figura 1: Arquitetura do processo em C. As duas camadas de entrada convergem para o mesmo módulo de IPC, o que as torna intercambiáveis do ponto de vista da aplicação Python.

Outro ponto de projeto é a detecção da `wiringPi` em tempo de compilação. A biblioteca só existe no Raspberry Pi; sem ela o binário compila do mesmo jeito (as funções de GPIO viram *stubs* que recusam a execução), o que permite exercitar o *parser* do teclado em qualquer máquina — e é exatamente disso que a suíte de testes da Seção 3 se aproveita.

1.2 Plano B: camada de teclado matricial

O risco identificado na semana 2 — a chapa de acrílico comprada era grande demais para o alcance dos ímãs, e a remontagem do tabuleiro consome tempo — motivou uma alternativa de contingência: em vez de mover a peça no tabuleiro instrumentado, o jogador **digita o lance** num teclado matricial 4×4 de baixo custo. Todo o resto do sistema permanece igual.

Impacto nos requisitos. O plano B não é neutro do ponto de vista da especificação, e vale declará-lo: com o teclado, **RF1** (“detecção das peças no tabuleiro físico e de seus movimentos”) deixa de ser satisfeito — o lance é informado, não detectado — e **RNF3** (robustez contra falsos positivos por esbarrões) perde o objeto, já que não há sensor a ser perturbado. **RF2**, **RF3** e **RNF2** continuam valendo integralmente, e são exatamente eles que o teclado preserva. A camada de *reed switches* continua implementada e é o caminho principal: o teclado é contingência para o caso de a montagem não fechar, não uma troca de escopo. A Seção 3.4 registra esse estado requisito a requisito.

O obstáculo é que o teclado tem apenas as letras A-D e o tabuleiro precisa de a-h. A solução adotada é o **toque repetido**: a mesma tecla pressionada duas vezes seguidas vale pela letra do bloco seguinte (A→a, AA→e), e um terceiro toque volta ao início, o que permite corrigir sem apagar. Não há temporizador envolvido — como o lance alterna coluna e fileira, duas letras seguidas só podem ser a mesma casa sendo redigitada, o que torna a desambiguação determinística.

Tabela 2: Exemplos de digitação de lances.

Lance	Teclas	Evento emitido
a2a4	A 2 A 4 #	a2:0,a4:1
e2e4	AA 2 AA 4 #	e2:0,e4:1
g1f3	CC 1 BB 3 #	g1:0,f3:1
Roque curto	AA 1 CC 1 # e depois DD 1 BB 1 #	e1:0,g1:1 e h1:0,f1:1

A tecla # confirma o lance e a * apaga a última tecla. O roque é digitado em dois lances (rei e depois torre), exatamente como já acontece no tabuleiro físico, o que reaproveita sem alteração o tratamento de roque em duas etapas já existente no Python.

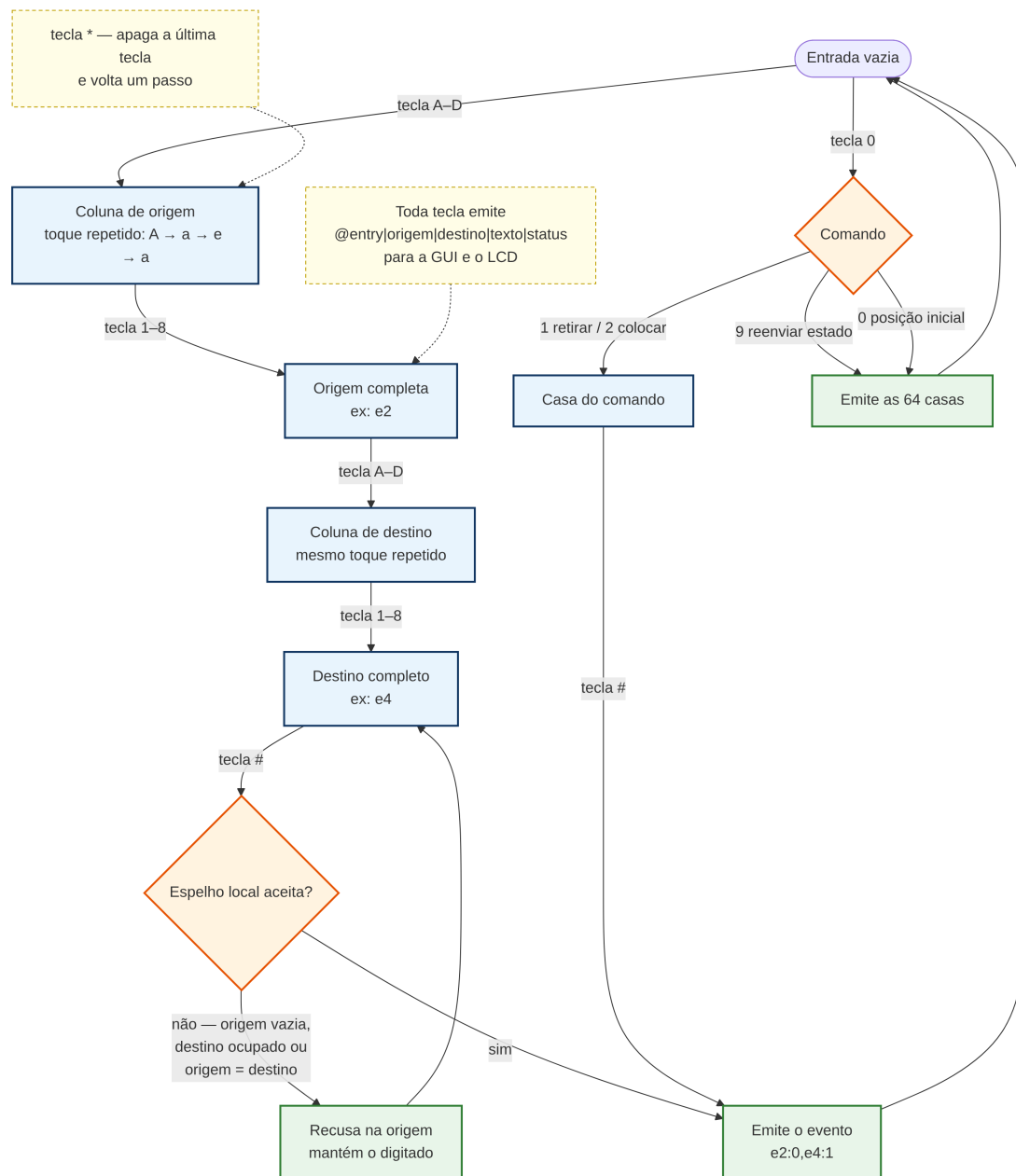


Figura 2: Fluxo de digitação de um lance na camada de teclado, incluindo os comandos e a recusa local.

O processo em C mantém um **espelho** de onde estão as peças do jogador e recusa na origem o que o tabuleiro físico também recusaria: mover a partir de uma casa vazia, mover para uma casa já ocupada por peça própria, ou origem igual ao destino. A legalidade do lance continua sendo decidida pelo Python — essa validação local só evita gerar eventos que o tabuleiro real jamais produziria. Como o espelho pode divergir do tabuleiro virtual (por exemplo, quando o oponente captura uma peça), há um conjunto de comandos com prefixo 0:

Tabela 3: Comandos do teclado (equivalentes a mexer numa peça sem fazer um lance).

Comando	Efeito	Quando usar
0 1 casa #	Retira a peça da casa	O oponente capturou uma peça
0 2 casa #	Coloca uma peça na casa	A aplicação pediu para repor uma peça
0 9 #	Reenvia o estado das 64 casas	Espelho e tabuleiro virtual divergiram
0 0 #	Volta o espelho à posição inicial	Recomeçar do zero

1.3 Extensão do protocolo IPC: linhas @entry

Na camada de *reed switches*, a peça na mão do jogador é visível pelos próprios sensores, e a GUI usa essa informação para destacar os destinos legais. Digitando o lance não há sensor nenhum — e sem alguma informação adicional o jogador digitaria às cegas até o #.

A solução foi estender o protocolo IPC com um segundo tipo de linha, que começa com @ — caractere que nenhuma casa do tabuleiro pode ter, de modo que qualquer leitor que só entenda casa:estado as ignora sem tropeçar:

```
| @entry|origem|destino|texto|status
```

Digitando AA2AA4# (lance e2e4), a sequência emitida é:

```
@entry|||Lance: A_|Digitando...
@entry|||Lance: E_|Coluna trocada
@entry|e2||Lance: E2_|Digitando...
@entry|e2||Lance: E2A_|Digitando...
@entry|e2||Lance: E2E_|Coluna trocada
@entry|e2|e4|Lance: E2E4|Digitando...
e2:0,e4:1
@entry|||Enviado e2e4
```

Com isso a GUI passa a destacar a casa de origem e os destinos legais da peça escolhida assim que as duas primeiras teclas formam uma casa, e a barra de status mostra o que já foi digitado. O destino digitado, ainda não confirmado, ganha uma borda âmbar. Só peças do próprio jogador são destacadas: apontar uma casa vazia ou uma peça do oponente é erro de digitação, e destacá-la sugeriria um lance inexistente.

1.4 Interface de configuração

Na revisão por pares da semana 2, o Grupo I sugeriu substituir a configuração por CLI por uma interface visual interativa, e o Grupo B pediu melhor sinalização de erros ao usuário. Ambas as sugestões foram atendidas nesta semana.

Tudo o que os alvos do Makefile escolhem por variáveis — modo de jogo, oponente, cor das peças, camada de entrada, tempo da *engine*, controle de tempo do Lichess — passou a ser escolhido também na própria interface, descrito num único módulo (app/launcher.py) e renderizado de duas formas:

uma janela *pygame* operada por teclado ou mouse, e uma lista numerada no terminal para quando não há *display* (acesso por SSH). Executar `python -m app.main` sem argumento nenhum abre o menu.

```
Tabuleiro de Xadrez Eletrônico
Stockfish (1s por lance) - peças brancas - entrada: mock (gui)
-----
Modo de jogo                      Stockfish local
Tempo por lance (s)                < 1.0 >
Cor das peças físicas              brancas
Entrada do tabuleiro              mock (sem hardware)
...
INICIAR PARTIDA
Compilar processo C (make board-input)
Sair
-----
Equivalente no terminal: make stockfish COLOR=white
Setas: escolher/alterar - Enter: editar ou iniciar - F5: compilar
```

Quanto à sinalização de erros, o menu **verifica as pré-condições antes de começar** e impede a partida enquanto houver impedimento: token do Lichess ausente, controle de tempo que a *Board API* recusa, processo em C ainda não compilado, Stockfish fora do PATH. Cada um vira uma mensagem explicativa no rodapé, em vez de um erro no meio da partida. O menu também compila o processo em C sem sair da interface e exibe o comando equivalente (`make ...`), de modo que a CLI continua disponível para quem desenvolve.

1.5 Ações durante a partida

Com a partida em andamento, a tecla `Esc` passou a abrir um menu de ações sobre o tabuleiro: reiniciar (modo Stockfish), abortar, oferecer ou aceitar empate, desistir, voltar ao menu de configuração ou sair. Antes, uma proposta de empate do oponente só podia ser respondida pelo site do Lichess.

Duas decisões de interface merecem registro. Primeiro, enquanto a partida está viva toda ação irreversível pede confirmação — num tabuleiro operado por toque, um encostão não pode custar a partida. Segundo, ao fim da partida o menu abre sozinho e sem confirmação, já com as opções que ainda fazem sentido, e “Voltar ao menu” devolve o jogador à configuração sem reabrir o programa.

2 Desenvolvimento de Hardware

A montagem do tabuleiro instrumentado avançou: cerca de **duas fileiras completas** de casas foram montadas com *reed switches* e diodos, o suficiente para confirmar em escala real o mapeamento de linhas e colunas e a detecção através da chapa. Ainda assim, a avaliação do grupo é que **não haverá tempo de concluir as 64 casas** dentro do cronograma da disciplina — o que confirma o risco levantado na semana 2 e justifica o plano B descrito acima.

Do lado do teclado matricial, a implementação foi **testada com o hardware de verdade**, reaproveitando a fiação já montada na bancada (a mesma do experimento 6): linhas nos pinos BCM 16, 20, 21, 26 e colunas em 19, 13, 6, 5. Para essa verificação o processo em C ganhou um modo de conferência de bancada:

```
$ ./build/board_input --input keypad --raw
[bancada] tecla 'A' (linha 0 = pino 16, coluna 3 = pino 19)
```

Nesse modo nenhum evento é enviado — ele só mostra em que interseção da matriz cada tecla foi lida, o que separa um problema de fiação de um problema de lógica. A conferência confirmou que o teclado da bancada é o 4×4 padrão — A B C D na quarta coluna, * 0 # D na última linha —, que é exatamente o mapeamento assumido em `keypad_config_default()`: nenhuma tradução de rótulos foi necessária.

A matriz de *reed switches* usa pinos distintos (linhas 4, 5, 6, 12, 13, 16, 19, 20, colunas 21, 22, 23, 24, 25, 26, 27, 17), de modo que as duas camadas podem coexistir fisicamente no mesmo Raspberry Pi.

3 Evidências de Testes Experimentais

3.1 Suíte automatizada

A suíte de testes ganhou uma quarta *suite*, `tests/test_keypad_layer.py`, com **30 verificações**. Ela não é um teste de unidade do Python: ela executa o binário em C de verdade (`build/board_input --input keypad --keys stdin`), digita as teclas na entrada padrão e confere os eventos que saem — ou seja, exercita o mesmo caminho de código do teclado real, trocando apenas a origem das teclas. Além disso, as linhas `@entry` são conferidas com o *parser* que a própria aplicação usa (`app.ipc_reader.parse_entry`), o que valida os dois lados do protocolo. A suíte é pulada, sem falhar, quando o binário não foi compilado.

Resultado da execução completa (`make test`):

```
=====
Todas as 4 suítes passaram.
```

3.2 Verificações da camada de teclado

A Tabela 4 reproduz as verificações executadas, agrupadas por objetivo. Todas passaram.

Tabela 4: Evidências da camada de teclado
(`tests/test_keypad_layer.py`).

Verificação	Teclas	Saída observada
Colunas: tecla simples e tecla repetida		

Verificação	Teclas	Saída observada
a2a4 com teclas simples	A2A4#	a2:0,a4:1
e2e4 com tecla repetida	AA2AA4#	e2:0,e4:1
b1c3 (cavalo)	B1C3#	b1:0,c3:1
g1f3 (cavalo)	CC1BB3#	g1:0,f3:1
d2h6 mistura os blocos	D2DD6#	d2:0,h6:1
Terceiro toque volta a a	AAA2AAA4#	a2:0,a4:1
<i>Correção com *</i>		
Apaga a fileira errada	AA2AA3*4#	e2:0,e4:1
Apaga tudo e redigita	AA2****A2A4#	a2:0,a4:1
# com entrada	AA2#	nada emitido
incompleta		
<i>Recusas locais (o que o tabuleiro físico também recusaria)</i>		
Origem vazia	AA5AA6#	nada emitido
Destino ocupado	A1A2#	nada emitido
Origem igual ao destino	AA2AA2#	nada emitido
<i>Comandos</i>		
0-1 retira a peça	01AA2#	e2:0
0-2 coloca a peça	02AA5#	e5:1
0-1 em casa já vazia	01AA5#	nada emitido
0-2 em casa já ocupada	02AA2#	nada emitido
<i>Sequências de partida</i>		
Espelho acompanha os lances	AA2AA4#AA4AA5#	e2:0,e4:1 e e4:0,e5:1
Roque curto em dois lances	AA2AA4# CC1BB3# BB1C4# AA1CC1# DD1BB1#	e1:0,g1:1 seguido de h1:0,f1:1
De pretas, e7e5	AA7AA5#	e7:0,e5:1
De pretas, e2 está vazia	AA2AA4#	nada emitido
<i>Reenvio do estado completo</i>		
0-9 reenvia as 64 casas	09#	1 linha com as 64 casas estado inicial correto
Fileiras 1 e 2 ocupadas	09#	
<i>Digitação em andamento (linhas @entry)</i>		
Cada tecla emite uma linha	AA2A	4 linhas @entry
Origem sai com a casa completa	AA2	origin='e2', text='Lance: E2_'
Casa incompleta não vira origem	AA2A	origin='e2' (inalterada)
Destino sai na quarta tecla	AA2AA4	target='e4'
# limpa a digitação	AA2AA4#	status='Enviado e2e4'
Comando não emite origem	01AA2	origin=None

Verificação	Teclas	Saída observada
* desfaz a origem Linhas @entry não contaminam os eventos	AA2* —	status='Apagado' canais separados

3.3 O que essas evidências cobrem

Vale destacar três resultados que respondem a riscos concretos do projeto:

- **O roque funciona pelo teclado.** A sequência de cinco lances produz exatamente os mesmos eventos que o tabuleiro físico produziria em duas etapas — ou seja, o plano B não perde nenhuma regra especial do xadrez.
- **A troca de camada é transparente.** Digitar AA2AA4# gera e2:0,e4:1, exatamente o que a matriz de *reed switches* emitiria para o mesmo lance. Nenhuma linha da camada Python precisou distinguir os dois casos.
- **As linhas @entry não contaminam o canal de eventos.** A verificação final confirma que um leitor que só entenda casa:estado continua funcionando — a extensão do protocolo é retrocompatível.

3.4 Rastreabilidade requisito × teste

A Tabela 5 atualiza o mapeamento apresentado na semana 1 (Seção 8.2 do primeiro relatório) com o que esta semana acrescentou, e registra o que ainda falta validar.

As duas pendências são de medição, não de implementação, e concentram o trabalho experimental da semana 4: RNF1 e RNF2 exigem instrumentar o caminho completo (tecla ou sensor → lance aplicado → quadro renderizado) no Raspberry Pi, e RNF3 depende das fileiras montadas para exercitar o *debouncing* com esbarrões reais.

4 Documentação e Diagramas

A documentação foi revisada e ampliada em cerca de 380 linhas no README, com seções novas para o menu de configuração, as opções durante a partida, o processo em C e suas duas camadas, o protocolo @entry e o roteiro de conferência da fiação do teclado (incluindo uma tabela de sintoma → causa provável). Todo módulo novo, em C e em Python, traz cabeçalho explicando a decisão de projeto que ele materializa, e não apenas o que ele faz.

Quanto aos diagramas, o fluxograma da semana 2 (arq_cpp) descrevia o laço de varredura herdado da prova de conceito em Arduino e não corresponde mais ao código. Ele permanece no repositório, referenciado pelo relatório daquela semana, mas foi substituído aqui por dois diagramas novos — mantidos como fonte em Mermaid (docs/diagramas/*.mmd) e renderizados por mmdc:

Tabela 5: Rastreabilidade entre requisitos e evidências, ao fim da semana 3.

Req.	Evidência	Estado
RF1	<i>Reed switches</i> : test_stockfish_loop (eventos de sensor → lances) e mock visual. Hardware: 2 fileiras montadas Teclado: 30 verificações de test_keypad_layer	Software ✓ · hardware parcial Não se aplica — o lance é informado, não detectado
RF2	test_stockfish_loop (<i>engine</i> falsa, <i>bestmove</i> aplicado) e test_lichess / test_challenge contra o servidor falso da <i>Board API</i>	✓
RF3	Mock com GUI (make mock); destaques de destino e barra de status verificados também pela camada de teclado (linhas @entry)	Visual ✓
RNF1	Precisão: 100% dos eventos digitados produziram o evento esperado (30/30). Latência < 200 ms	Precisão ✓ · latência pendente
RNF2	Instrumentação de <i>timestamps</i> na GUI	Pendente
RNF3	Ressincronização (0-9, 0-0) e recusa local de eventos impossíveis, verificadas na suíte	Software ✓ · <i>debouncing</i> em hardware pendente

- `arq_c.mmd` — arquitetura modular do processo em C, mostrando as duas camadas convergindo para o mesmo módulo de IPC (Figura 1);
- `fluxo_keypad.mmd` — máquina de digitação de um lance, incluindo comandos e recusa local (Figura 2).

Este relatório passou a ser escrito diretamente em $\text{\LaTeX}$ (`docs/Relatorio-3.tex`), em vez de Markdown convertido por Pandoc. O alvo `make pdf3` gera os diagramas e compila o documento com `xelatex`, tudo dentro do ambiente reprodutível do Nix.

5 Estruturação do Relatório Final

O relatório final consolidará os relatórios semanais na estrutura abaixo, herdada do relatório da semana 1 e ajustada ao que o projeto se tornou. A coluna de estado indica o que já existe escrito e o que falta.

O limite de 20 páginas de conteúdo (sem referências e anexos) é a restrição que orienta a consolidação: os relatórios semanais somados excedem esse total, de modo que as seções de implementação serão condensadas em torno das decisões de projeto, e os detalhes de operação — opções de linha de comando, tabelas de teclas, protocolo IPC — ficam no README, referenciado a partir do relatório.

6 Entrega e Próximos Passos

A terceira *release* no GitHub marca o estado descrito aqui: processo em C modular com duas camadas de entrada, interface de configuração e de ações, suíte com quatro conjuntos de testes e documentação revisada.

Para a semana 4:

- medir a latência ponta a ponta (tecla ou sensor até o lance aplicado na *engine*), o requisito de < 200 ms definido na semana 1 — é a evidência experimental que ainda falta;
- fechar o que for possível da montagem do tabuleiro e medir a taxa de acerto da detecção nas fileiras já montadas;
- teste de aceitação: partidas completas contra o Stockfish e contra a IA do Lichess pelo teclado, do início ao xeque-mate;
- consolidar o relatório final na estrutura da seção anterior, dentro do limite de 20 páginas, com as citações no texto em norma ABNT;
- registrar em `docs/figuras/` o material fotográfico da bancada e da montagem, como evidência das validações de hardware descritas na Seção 2;
- recolher as *branches* `plano_b` e `interface`, já integradas por *pull request*, deixando apenas `main` na entrega final.

Tabela 6: Estrutura prevista para o relatório final.

Seção	Conteúdo	Estado
1	Motivação e visão geral	Semana 1, revisar
2	Objetivos	Semana 1, revisar o escopo de hardware
3	Requisitos funcionais e não funcionais	Semana 1 (RF1–RF3, RNF1–RNF3); revisar à luz do plano B
4	Ferramentas utilizadas	Atualizar: C no lugar de C++/Arduino, teclado 4×4
5	Metodologia de desenvolvimento	Semana 1; acrescentar o plano B como decisão de risco
6	Arquitetura física	Semana 1; atualizar com o teclado e a placa efetivamente usada
7	Arquitetura de software (estática e comportamental)	Semana 1 + diagramas novos da semana 3
8	Implementação: camada Python	Semanas 1–3
9	Implementação: processo em C e camadas de entrada	Semana 3 (Seções 1.1–1.3)
10	Interface com o usuário	Semana 3 (Seções 1.4–1.5)
11	Hardware: tabuleiro e teclado	Semanas 2–3, pendente o fechamento da montagem
12	Testes: casos planejados, resultados e rastreabilidade	Semana 3 (Seção 3); falta a medição de latência
13	Revisão por pares e evolução do projeto	Semana 2 (Issues nos repositórios dos Grupos P e S)
14	Conclusões, dificuldades e lições	A escrever na semana 4
15	Referências (ABNT)	Semana 1; acrescentar as citações no texto